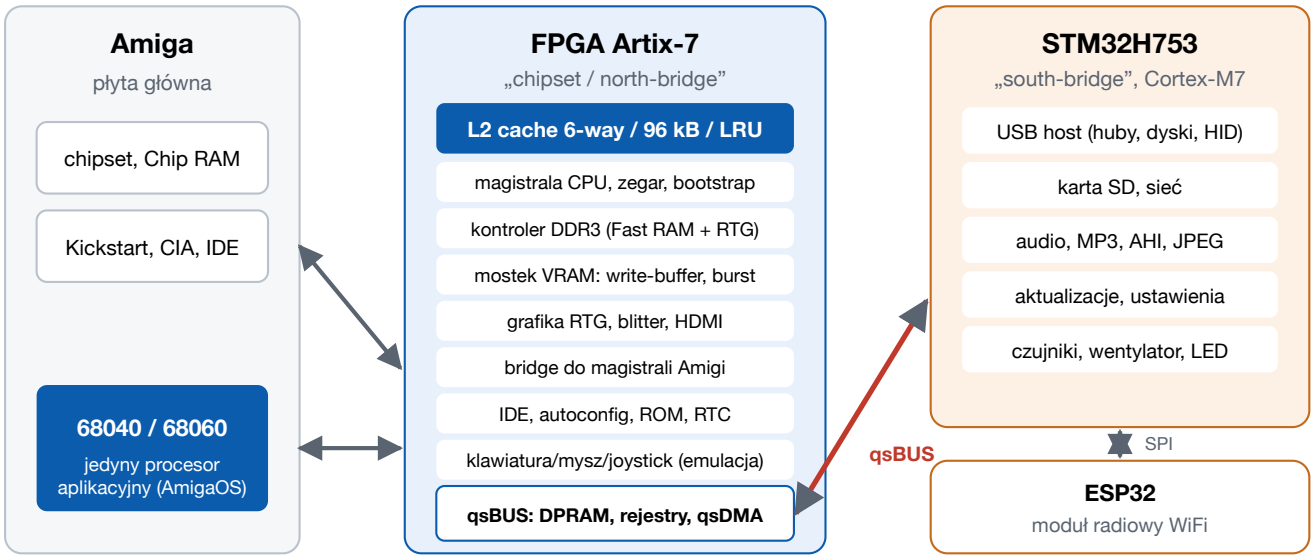
MATERIAŁ DO PREZENTACJI · CS-LAB

Karty Warp architektura i firmware 3.0

Jak zbudowane są karty turbo Warp dla Amigi: co robi układ FPGA, co robi mikrokontroler STM32H7, jak oba układy rozmawiają ze sobą — oraz co nowego przynosi pakiet firmware 3.0 i nowe aplikacje WarpDashboard, WarpStressTest i WarpPDFView.



Rys. 1. Trzy „mózgi” karty Warp. Programy użytkownika wykonuje wyłącznie procesor 68k.

Opracowanie na podstawie kodu źródłowego i historii zmian projektu (firmware STM32, logika FPGA, oprogramowanie AmigaOS).
Stan na październik 2026, pakiet firmware 3.0.
Liczby i pomiary pochodzą z testów na sprzęcie opisanych w dzienniku zmian.

1. Rodzina kart Warp

Warp to rodzina kart turbo dla klasycznych Amig z procesorami Motorola 68040 i 68060. Wszystkie modele mają tę samą architekturę: procesor 68k, układ FPGA Xilinx Artix-7 (XC7A35T) z pamięcią DDR3, mikrokontroler STM32H753 oraz moduł WiFi ESP32. Różnią się procesorem, magistralą płyty głównej, do której się podłączają, i wyposażeniem portu USB.

Model	Komputer	CPU	Port USB	Uwagi
Warp560	Amiga 500	68060	Full-Speed (12 Mb/s)	bridge do magistrali 68000
Warp1240	Amiga 1200	68040	Full-Speed	
Warp1260	Amiga 1200	68060	Full-Speed	
Warp1260X	Amiga 1200	68060	High-Speed (480 Mb/s, PHY ULPI)	SDRAM przy STM32
Warp7060	Amiga 3000/4000	68060	High-Speed (PHY ULPI)	nadajnik HDMI, SDRAM przy STM32, dwukierunkowy mostek do magistrali płyty głównej z obsługą DMA z kart Zorro i kontrolerów SCSI

Z punktu widzenia użytkownika karta to znacznie więcej niż szybszy procesor: Fast RAM w pamięci DDR3, grafika RTG z wyjściem HDMI, pamięć podręczna L2, kontroler IDE, karta SD jako dysk, porty USB (klawiatura, mysz, gamepady, pendrive'y, dyski, CD-ROM, stacja dyskietek, karta sieciowa), WiFi, kodek audio z dekoderem MP3 i sterownikiem AHI, zegar czasu rzeczywistego, sterowanie wentylatorem i podświetleniem RGB. Każdą z tych funkcji realizuje jeden z dwóch układów pomocniczych — FPGA albo STM32 — i ten podział nie jest przypadkowy.

2. Podział ról: jeden procesor, dwa „mostki"

Dobrą analogią jest klasyczny komputer PC z przełomu wieków. Był w nim procesor, a obok niego dwa układy chipsetu: **north-bridge**, który obsługiwał rzeczy szybkie i ściśle zsynchronizowane z procesorem (magistrala CPU, kontroler pamięci, grafika), oraz **south-bridge**, który obsługiwał wolniejsze, ale skomplikowane protokolarnie urządzenia wejścia/wyjścia (USB, dyski, dźwięk, sieć, zegar, BIOS). W karcie Warp jest dokładnie tak samo:

Układ	Rola	Skala czasu i charakter pracy
68040 / 68060	procesor aplikacyjny	Wykonuje AmigaOS i wszystkie programy użytkownika. To jedyny procesor w systemie, dla którego pisze się aplikacje.
FPGA Artix-7	„north-bridge" + chipset graficzny	Nanosekundy. Logika sprzętowa pracująca w takt zegara magistrali CPU: każdy cykl odczytu i zapisu procesora przechodzi przez FPGA.
STM32H753	„south-bridge", koncentrator I/O	Mikro- i milisekundy. Oprogramowanie (firmware) obsługujące protokoły: USB, SD, sieć, audio, aktualizacje.
ESP32	modem WiFi	Moduł radiowy za STM32; nie ma bezpośredniego połączenia z Amigą.

Zasada podziału jest prosta. To, co musi odpowiedzieć procesorowi 68k w ciągu jednego lub kilku taktów zegara magistrali, musi być sprzętem — robi to FPGA. To, co jest protokołem z wieloma stanami, wyjątkami, timeoutami i ponowieniami (USB, system plików karty SD, WiFi, dekodowanie MP3), byłoby w logice FPGA ogromne i trudne do poprawiania — robi to oprogramowanie w STM32. Oba układy łączy wewnętrzne łącze **qsBUS** (rozdział 5).

3. Co robi FPGA

Projekt FPGA (katalog `hdl/`) to jeden wspólny projekt dla wszystkich pięciu kart — ok. 60 plików VHDL współdzielonych między modelami — uzupełniony o elementy specyficzne dla danej płyty. Wszystko, co jest opisane poniżej, działa sprzętowo, bez udziału jakiegokolwiek procesora.

Procesor i magistrale

- Obsługa magistrali 68040/68060: odpowiedzi na cykle, potwierdzenia transferu, burst, przerwania.
- Generator zegara CPU na PLL, przestrajany w locie (częstotliwość ustawiana z firmware).
- Bootstrap 68060 i mapowanie ROM: Kickstart z pamięci flash karty (MapROM), monitor overlay.
- **Bridge do magistrali płyty głównej** (68000 w A500, 68020/030 w A1200/A3000) z arbitrażem — dostęp do Chip RAM, chipsetu i CIA.
- Dekodowanie adresów i AutoConfig.

Pamięć

- Kontroler pamięci DDR3 i interkonekt, który dzieli ją między Fast RAM procesora, pamięć grafiki RTG i transfery DMA z STM32.
- **Pamięć podręczna L2**: 96 kB, 6-way, wymiana LRU, write-back, sprzętowe liczniki trafień (opis niżej).
- **Osobny mostek do pamięci VRAM** z buforem zapisu (write-buffer) i transferami burst (opis niżej).

Grafika RTG i wideo

- Bufor ramki w DDR3, zapisywany przez CPU przez osobny mostek VRAM.
- Blitter sprzętowy: wypełnienia, kopiowanie bloków, linie (Bresenham), konwersja wzorców.
- Sprzętowy kursor (sprite), paleta CLUT, skalowanie z interpolacją, generator timingów.
- Enkoder TMDS — wyjście HDMI/DVI wprost z FPGA (w Warp7060 przez nadajnik HDMI).
- Odbiornik obrazu z WarpedVision i nakładka OSD.

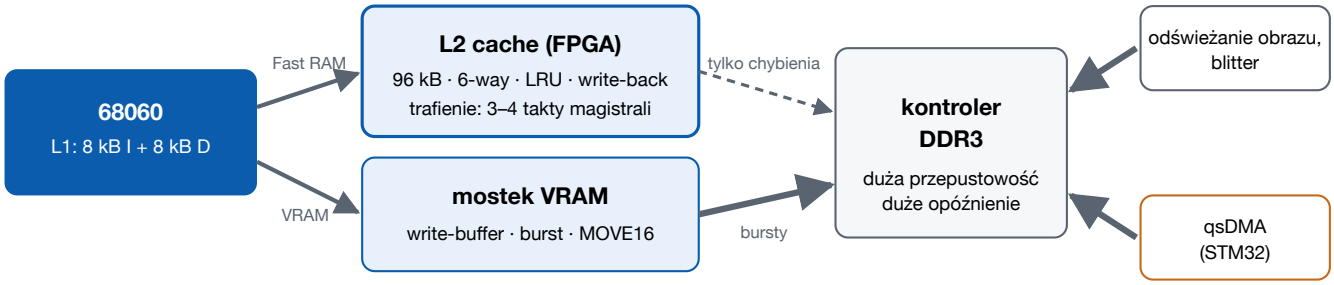
Peryferia Amigi

- Kontroler IDE/ATA.
- Emulacja zegara RTC.
- Emulacja klawiatury, myszy i joysticka Amigi — rejestry, do których STM32 wpisuje zdarzenia z urządzeń USB; od 3.0 z 16-elementową kolejką kodów klawiszy.
- Dekoder S/PDIF (wejście audio).
- Programowanie własnej pamięci flash (bitstream, Kickstarty).

Pamięć DDR3: duża przepustowość, duże opóźnienie

Fast RAM, pamięć grafiki i bufory DMA leżą w jednej kości DDR3, obsługiwanej przez kontroler w FPGA. Kontroler DDR3 operuje słowami 128-bitowymi i jest zoptymalizowany pod **długie transfery burst**: gdy już otworzy wiersz pamięci, dane płyną bardzo szybko. Dlatego świetnie sprawdza się w grafice — odświeżanie obrazu czyta całe linie bufora ramki, blitter kopiuje duże bloki, a qsDMA przesyła całe sektory. Każdy **pojedynczy** dostęp płaci jednak pełny koszt: przejście do domeny zegara kontrolera, kolejkę komend, otwarcie wiersza, opóźnienie CAS i drogę powrotną. To opóźnienie (latency) jest wielokrotnie dłuższe niż kilka taktów, w których procesor 68k oczekuje odpowiedzi.

A 68040/68060 robi właśnie to, czego DDR3 nie lubi: pojedyncze odczyty długich słów i wypełnienia 16-bajtowych linii własnej pamięci podręcznej (68060 ma 8 kB cache danych i 8 kB cache instrukcji). Bez dodatkowej warstwy każde chybienie w cache procesora czekałoby na pełne opóźnienie DDR3, a szybki procesor w dużej mierze stałby w miejscu.



Rys. 2. Droga procesora do pamięci DDR3: Fast RAM przez cache L2, pamięć grafiki przez osobny mostek VRAM.

Cache L2: rozwiązanie problemu opóźnienia

Dlatego w FPGA, przed kontrolerem DDR3, działa pamięć podręczna drugiego poziomu: **96 kB, 6-way, z polityką wymiany LRU**, liniami 128-bajtowymi i zapisem write-back. Jedno chybienie sprowadza z DDR3 od razu 128 bajtów — osiem 16-bajtowych linii cache procesora — w jednym długim burście, czyli dokładnie tak, jak DDR3 pracuje najwydajniej; kolejne odczyty z sąsiednich adresów trafiają już w L2. Pracuje w takt zegara magistrali procesora i odpowiada na trafienie w 3–4 takty, bez udziału kontrolera DDR3. Do pamięci DDR3 trafiają tylko chybienia, i to w postaci, którą DDR3 obsługuje najlepiej: wypełnień i zapisów całych linii.

- **Skuteczność:** w wielu aplikacjach obecna implementacja L2 osiąga **ponad 90% trafień** — ponad 9 na 10 dostępów procesora, które nie mieszczą się w jego własnym cache, w ogóle nie czeka na DDR3.
- **LRU zamiast round-robin:** o ok. 24% mniej chybień i ok. 2,9% więcej klatek w Quake (Warp1260, 68060 @ 107 MHz).
- **Wczesne potwierdzenie odczytu** (early read ack): trafienie w 3 zamiast 4 takty magistrali. Od 3.0 na wszystkich kartach, przełączane w locie z poziomu firmware.
- Sprzętowe liczniki dostępów, trafień, chybień i zapisów write-back — widoczne w WarpDiag i WarpTool, co pozwala zmierzyć skuteczność L2 w konkretnym programie.

Osobny mostek VRAM dla zapisu grafiki

Pamięć grafiki ma inne potrzeby niż Fast RAM: procesor głównie do niej **zapisuje**, często dużymi blokami. Dlatego zapis do VRAM idzie całkowicie osobną ścieżką — mostkiem z buforem zapisu (write-buffer), który potwierdza zapis procesorowi od razu, a dane składa w transfery burst do DDR3. Jest on zoptymalizowany pod instrukcję **MOVE16** procesora 68040/68060, która przesyła 16 bajtów naraz — tak uzyskuje się największą przepustowość przy kopiowaniu obrazu z Fast RAM do pamięci grafiki. Pojedynczy zapis zajmuje 3 takty magistrali, a burst 6.

Strona FPGA łącza qsBUS

Interfejs QSPI typu slave i interkonekt, który rozprowadza dostęp STM32 do zasobów FPGA: dwuportowa pamięć **DPRAM** (okno wymiany danych widoczne jednocześnie dla 68k i dla STM32), blok rejestrów **csDPREG** (przerwania i statusy dla Amigi — rozdział 5), silnik **qsDMA** przenoszący dane pomiędzy STM32 a pamięcią DDR3 (z trybem modulo i zamianą bajtów), rejestry konfiguracji systemu (reset, PLL, liczniki L2, wczesne potwierdzenie L2) oraz generowanie przerwania INT2 dla procesora Amigi.

qsDMA nie zabiera procesorowi magistrali. W klasycznej Amidze DMA z karty rozszerzeń przejmuje szynę i procesor w tym czasie czeka. W karcie Warp qsDMA jest jednym z równorzędnych klientów interkonektu pamięci w FPGA, obok procesora 68k i grafiki — nie przejmuje szyny od 68k. Transfer danych, np. z dysku USB do bufora w Fast RAM, odbywa się równocześnie z pracą procesora, który ma w tym czasie normalny dostęp do pamięci poza obszarem objętym transferem DMA.

Przykład z 3.0: ile da się wycisnąć z magistrali płyty głównej

Dostęp do Chip RAM zależy od tego, jak bridge w FPGA wpasowuje się w rytm magistrali płyty głównej. W 3.0 przeanalizowano go analizatorem stanów logicznych (ok. 1,5 mln cykli magistrali A500) i modelem symulacyjnym, a następnie skrócono ścieżkę bridge'a na każdej karcie:

Karta	Przed	Po (3.0)	Komentarz
Warp560 (A500)	2,8 MB/s zapis	**3,5 MB/s**	praktycznie fizyczny limit (ceiling): 2 cykle 68000 na długie słowo, 3,55 MB/s
Warp1240 (A1200)	4,7 MB/s zapis	**7,0 MB/s** od 47 MHz	ceiling magistrali: 7,09 MB/s
Warp1260 (A1200)	7 MB/s osiągane już od 50 MHz, a nie dopiero od 100 MHz		

4. Co robi STM32 — „south-bridge" karty

STM32H753 to mikrokontroler z rdzeniem ARM Cortex-M7 taktowanym 400 MHz, z ok. 1 MB wewnętrznej pamięci SRAM i 2 MB pamięci flash na firmware. Działa pod kontrolą systemu czasu rzeczywistego ThreadX; firmware to kilkanaście wątków, z których każdy obsługuje jedną grupę urządzeń. Z punktu widzenia Amigi STM32 jest **dostawcą usług**: AmigaOS widzi go przez standardowe interfejsy systemu — urządzenia (`.device`), sterowniki AHI i MHI, datatype — oraz przez `cswarp.library` .

Obszar	Zadania STM32	Widoczne w AmigaOS jako
USB host	Pełny stos hosta USB (USBX + własna warstwa sterownika kontrolera): huby (również kaskadowe), urządzenia LS/FS za hubem HS, odzyskiwanie po błędach, telemetria.	—
Klawiatura, mysz, pady	Raporty HID → kody klawiszy, ruch myszy, stan joysticka; dowolny pad HID dzięki analizie deskryptora raportu, mapowanie przycisków, auto-ogień.	natywna klawiatura, mysz i joystick Amigi (przez FPGA)
Pamięci masowe USB	Mass Storage (BOT, SCSI), dwa dyski jednocześnie, CD/DVD z dowolnym rozmiarem bloku, stacja dyskietek USB (UFI) z buforem całego cylindra.	`warpUSBDisk.device` , `warpFloppy.device`
Karta SD	Kontroler SDMMC, często dysk systemowy.	`warpSD.device`
Sieć	WiFi przez ESP32 (SPI, kontrola przepływu), Ethernet USB (ASIX AX88772), statystyki.	`warpNET.device` (SANA-II)
Audio i MP3	Kodek audio (44,1 kHz), miks dźwięku Amigi, MP3 i AHI; programowy dekoder MP3 z korektorem i konwersją częstotliwości.	sterownik MHI
AHI (nowość w 3.0)	Odbiór strumienia AHI z pierścienia w DPRAM, interpolacja do 44,1 kHz, miks z pozostałymi źródłami.	`warp.audio`
JPEG (nowość w 3.0)	Sprzętowy dekoder JPEG mikrokontrolera + konwersja kolorów DMA2D, wynik wysyłany kafelkami przez qsDMA.	`warpjpeg.datatype` , loader ImageFX
System karty	Bootloader i aktualizacje (pakiety `.pk` , zapis bitstreamu FPGA, firmware ESP32), ustawienia w podtrzymywanej pamięci, konfiguracja zegara CPU i L2, wybór Kickstartu, reset fabryczny, konfiguracja nadajnika HDMI.	WarpTool, WarpDiag, Boot Menu
Nadzór sprzętu	Temperatury i napięcia (ADC), wentylator, podświetlenie RGB, statystyki obciążenia.	WarpDashboard, WarpDebug

STM32 to mikrokontroler, a nie procesor aplikacyjny

Na prezentacjach często pada pytanie: *„czy będzie można pisać własne programy na ARM w karcie Warp?”* Odpowiedź brzmi: **nie — i wynika to z natury układu, a nie z braku chęci.**

- **To inna klasa układu.** STM32H7 to mikrokontroler (Cortex-**M**), a nie procesor aplikacyjny (Cortex-**A**, jak w Raspberry Pi czy w telefonie). Nie ma jednostki MMU ani pamięci wirtualnej, nie uruchomi Linuksa ani innego systemu ogólnego przeznaczenia, ma ok. 1 MB wewnętrznej pamięci RAM. Firmware to jeden monolityczny program w pamięci flash, a nie system, do którego „doinstalowuje się” aplikacje.
- **Jego czas jest już w całości zajęty.** Kontroler USB w STM32 wymaga, by procesor osobiście obsłużył praktycznie każdą transakcję na magistrali (rozdział 6). Gdy w 3.0 jeden wątek ponawiał zapytanie do stacji dyskietek 30 000 razy na sekundę, mysz na tym samym hubie zaczynała się zacinać. Obcy kod konkurujący o procesor oznaczałby zgubione klawisze, przerwy w dźwięku i zawieszone dyski.
- **Brak izolacji = ryzyko dla danych.** STM32 obsługuje dyski Amigi i transfery DMA do jej pamięci. Błąd w kodzie użytkownika bez ochrony pamięci może zawiesić cały komputer albo uszkodzić dane na dysku.
- **W PC też nikt nie programuje south-bridge’a** ani kontrolera w dysku SSD — choć to również procesory. Programy pisze się dla CPU.

Co w zamian? Moc STM32 jest udostępniana programom Amigi w postaci **usług**, dodawanych w kolejnych wersjach firmware: dekodowanie MP3 (MHI), miksowanie i odtwarzanie AHI, dekodowanie JPEG (datatype), sieć, dyski. Każdy program na Amigę może z nich korzystać przez standardowe API AmigaOS lub `cswarp.library`. Pomysły na kolejne takie usługi są mile widziane.

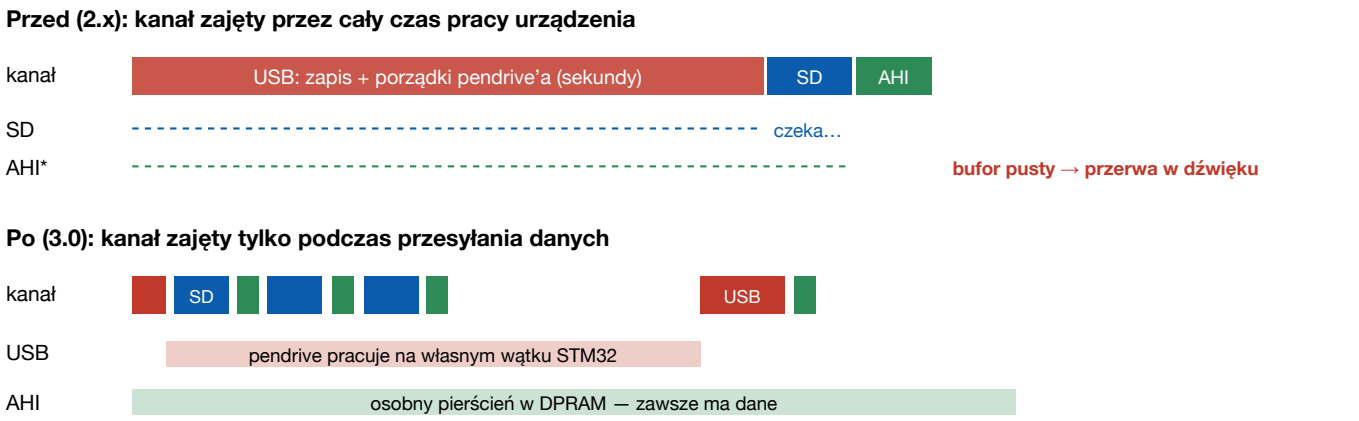
5. qsBUS: przeprojektowanie kanału komunikacji (async redesign)

qsBUS to wewnętrzne łącze pomiędzy STM32 a FPGA: nowoczesna, szybka 4-bitowa magistrala QSPI taktowana **50 MHz w trybie DDR** (dane na obu zboczach zegara), czyli **50 MB/s**. Przy takiej częstotliwości o poprawności decydują pojedyncze nanosekundy, dlatego timing łącza był dopracowywany z oscyloskopem, aż transfer stał się w 100% poprawny. Przechodzi przez nie absolutnie wszystko, co Amiga i STM32 mówią sobie nawzajem: każda komenda, wszystkie dane dyskowe i sieciowe, dźwięk AHI, rejestry klawiatury i myszy. Po stronie FPGA łącze prowadzi do pamięci DPRAM (okno komend widoczne dla 68k), do rejestrów sterujących oraz — przez qsDMA — wprost do pamięci DDR3, tak że dane z dysku mogą trafić bezpośrednio do bufora programu na Amidze. Interkonekt pamięci pozwala qsDMA pracować bez przejmowania szyny od procesora 68k: dane z dysku USB płyną do pamięci, a procesor w tym samym czasie normalnie korzysta z pamięci poza obszarem transferu.

Problem: jeden kanał, jedna kolejka

Do wersji 2.x komunikacja była synchroniczna. Amiga wpisywała komendę do DPRAM, a wątek komunikacyjny w STM32 wykonywał ją **do końca** i dopiero wtedy odpowiadał. Przez cały ten czas kanał był zajęty, a wszystkie inne urządzenia czekały:

- tani pendrive, który w środku zapisu robił wewnętrzne porządki, potrafił na kilka sekund zatrzymać kartę SD — zwykle dysk systemowy;
- komenda stacji dyskietek blokowała wszystko nawet do 20 s, rozpędzanie płyty CD — do 8 s;
- w takim modelu nie dało się poprawnie zrealizować **dźwięku AHI**: strumień audio musi płynąć bez przerwy, a każdy dłuższy transfer dyskowy zatrzymałby go na sekundy. Wewnętrzne wersje testowe z AHI na wspólnym kanale potwierdziły to słyszalnymi przerwami — dlatego publiczne wersje 2.x nie miały AHI, a sterownik ukazuje się dopiero razem z nowym kanałem w 3.0.



Rys. 3. Ten sam scenariusz przed i po przeprojektowaniu kanału (schematycznie). * AHI na starym kanale istniało tylko w wewnętrznych wersjach testowych.

Rozwiązanie w 3.0: model „zleć i zostań powiadomiony”

- **Krótkie zlecenie.** Amiga wysyła zlecenie (np. „odczytaj 200 sektorów z jednostki 1”) i natychmiast zwalnia kanał.
- **Osobny wątek i bufor dla każdej jednostki** — SD, dwa dyski USB, CD-ROM, stacja dyskietek. Żadna nie blokuje innych.
- **Powiadomienie sprzętowe.** W FPGA powstał nowy blok rejestrów **csDPREG** (64 słowa): dla każdej jednostki słowo statusu, licznik bajtów i bit przerwania. STM32 zapisuje je w ściśle określonej kolejności (licznik, status, flaga), więc gdy 68k dostaje przerwanie INT2, dane są już na miejscu. Odczyty z 68k są „migawką” — długie słowo nigdy nie jest rozerwane.
- **Strumieniowanie.** Duży odczyt jest dzielony na połówki bufora (2×128 KB na kartach z SDRAM): STM32 czyta z urządzenia część k+1, gdy Amiga odbiera część k. Zapis działa odwrotnie.
- **AHI poza kanałem komend.** Górne 4 KB DPRAM to pierścień audio z własnymi wskaźnikami zapisu/odczytu; STM32 opróżnia go w rytmie miksera. Ruch dyskowy i sieciowy nie może już spowodować przerwy w dźwięku.
- **„Dzwonki”** (doorbells): przerwanie, gdy bufor MP3 ma miejsce lub przyszła ramka sieciowa — koniec odpytywania.
- **Odporność na reset Amigi:** zlecenie przyjęte przed resetem nie może „odpowiedzieć” nowo uruchomionemu systemowi.

Zmiana objęła trzy warstwy naraz: logikę FPGA (rejestry i przerwania), firmware STM32 (nowy silnik zleceń, ok. 2200 zmienionych linii) i sterowniki AmigaOS — wszystkie trzy części są w pakiecie firmware i instalują się razem. Nowe polecenie `qspiStats` pokazuje obciążenie łącza: kopiowanie między szybką kartą SD a szybkim dyskiem USB zajmuje je w ok. 80%, a dekodowanie zdjęcia 12 Mpx w ok. 92% — łącze wykonuje realną pracę, zamiast czekać.

6. Walka z USB: kontroler, który nie jest EHCI

Obsługa USB okazała się największym pojedynczym wyzwaniem projektu. W cyklu wydania 2.282 na samo USB przypadło ok. **180** commitów i około trzech czwartych całego czasu pracy. Żeby zrozumieć dlaczego, trzeba porównać kontroler w STM32 z tym, co znamy z komputerów PC.

Kontroler EHCI/xHCI w PC

- Sterownik buduje w pamięci **listy deskryptorów** transferów; kontroler sam je przegląda.
- Ponawianie po NAK, przełączanie DATA0/DATA1, planowanie mikroramek i transakcji dzielonych (split) dla urządzeń LS/FS za hubem HS — **sprzętowo**.
- Procesor dostaje jedno przerwanie po zakończeniu całego transferu.

OTG_HS/OTG_FS w STM32H7

- Rdzeń Synopsys DesignWare USB OTG, projektowany głównie jako **urządzenie**; tryb hosta to 16 „kanałów”.
- Każdy transfer programuje oprogramowanie; kanał zatrzymuje się i zgłasza przerwanie przy NAK, błędach, zakończeniu. O ponowieniu decyduje **procesor**.
- Transakcje dzielone (SSPLIT/CSPLIT) i rytm mikroramek dla urządzeń okresowych — **w oprogramowaniu**.

W praktyce procesor STM32 jest częścią kontrolera USB. Do tego warstwa sterownika dostarczana przez producenta (HAL) oraz stos USBX zawierały błędy, które ujawniają się dopiero po godzinach obciążenia, z hubami, z urządzeniami LS/FS za hubem High-Speed i przy DMA. Najważniejsze znalezione i naprawione problemy:

Objaw	Przyczyna
Zapis na dysk USB „zamarza” (wedge)	Warstwa HAL po błędzie transmisji na kanale OUT ślepo reaktywowała kanał zamiast go zatrzymać i zgłosić błąd. Ten jeden charakterystyczny zestaw objawów wystąpił w 32 z 32 zarejestrowanych awarii.
Losowe uszkodzenia pamięci, awarie	Przy krótkim odczycie (13-bajtowy status SCSI) rozmiar transferu jest zaokrąglany do pełnego pakietu — DMA zapisywało cały pakiet za koniec bufora. Niewidoczne dla pułapek debuggera, bo pisze DMA, nie CPU.
Zablokowane potoki, zgubione dane sieci	Pakiety zerowej długości (ZLP) nie są wliczane do licznika pakietów — przełącznik DATA0/DATA1 odwracał się i potok się blokował.
Całkowite zamrożenie firmware	Ponowne zgłoszenie transferu okresowego, który był w toku, tworzyło w USBX listę wskazującą samą na siebie.
Dysk „martwy” po odzyskaniu	Po resecie w kolejce zostawały stare dane i stary status (CSW); trzeba je najpierw odebrać i odrzucić.
100% CPU, zacinająca się mysz	„Burze NAK”: urządzenie odpowiada „jeszcze nie”, a host pyta ponownie nawet 189 000 razy na sekundę; kanały split nie miały żadnego ograniczenia tempa.
Zgubione puszczenia klawiszy	Druga połowa transakcji split nie była wysyłana w następnej mikroramce, jak wymaga specyfikacja — hub zdążył wyrzucić dane.
Port HS sam się wyłącza	Spadek napięcia na szynie zasilania A1200 wyzwalał komparator VBUS, a rdzeń sam zdejmował zasilanie portu („fantomowe przeciążenie”).
Uszkodzone połówki słów	Bufory odbiorcze DMA w zewnętrznej SDRAM — przeniesione do wewnętrznej SRAM.
Cicho obcięte transfery	Limit liczby pakietów na kanał obcinał długie transfery zamiast zgłaszać błąd.

Metoda: pomiar zamiast zgadywania

Większości tych błędów nie da się zobaczyć debuggerem. Powstały więc własne narzędzia: **ślad zdarzeń USB** zapisywany przez firmware w pierścieniu, który przetrwa awarię i można go zapisać na dysk z poziomu Amigi bez debuggera; **analizator przechwyceń** z analizatora stanów logicznych, który dekoduje USB LS/FS aż do numeru sektora i nakłada na to ślad firmware; liczniki telemetrii oraz wielogodzinne testy obciążeniowe. Pozwoliło to też uczciwie oddzielić błędy firmware od usterek sprzętu testowego (np. wadliwa stacja dyskietek, która tak samo psuła zapis pod Windows).

Efekty: testy, które wcześniej kończyły się błędem po kilku minutach, przechodzą godzinami na Warp1260X i Warp7060; odzyskiwanie urządzenia skrócone z ~70 s do ~43 s, z resetem tylko jednego portu huba; odczyt pliku 1,3 MB

ze stacji dyskietek USB z **364 s** do **47 s** (szybciej niż ten sam napęd pod Linuksem); Ethernet USB z ~8 do ~28 Mb/s; 243 fałszywe alarmy watchdoga w jednym teście — wyeliminowane.

7. Co nowego w pakiecie firmware 3.0

Wersja 3.0 to pierwsza wersja z przeprojektowanym kanałem komunikacji (rozdział 5). Wokół tej zmiany powstało kilkadziesiąt poprawek i nowych funkcji — w firmware STM32, w logice FPGA wszystkich pięciu kart i w oprogramowaniu AmigaOS.

Wydajność i stabilność

- **Urządzenia niezależne od siebie:** SD, dwa dyski USB, CD-ROM i dyskietka pracują równolegle; duże transfery strumieniowane.
- **Szybszy Chip RAM** na Warp560, Warp1240 i Warp1260 (rozdział 3).
- **Wczesne potwierdzenie odczytu L2** na wszystkich kartach, przełączane w locie i zapamiętywane.
- Poprawiony zegar systemowy firmware: wszystkie wewnętrzne timeouty były o 20% dłuższe niż zamierzone.

Klawiatura, mysz, USB

- **Koniec gubionych klawiszy:** kolejka 16 kodów w FPGA, wydawanych w tempie, w jakim AmigaOS je odbiera — wyścig usunięty, a nie zawężony.
- **Mysz nie zacina się** podczas pracy stacji dyskietek na tym samym hubie: ruch przez hub mniejszy 60-krotnie, obciążenie przerwaniami USB poniżej 1/3.
- Klawiatura za hubem nie gubi już raportów (poprawne transakcje split).

Sieć

- **WiFi z kontrolą przepływu:** ramki nie są już gubione, gdy radio jest zajęte — każda zgubiona kosztowała retransmisję i połowę prędkości.

Audio

- **Nowość: sterownik AHI** `warp.audio`**.** Dźwięk AHI (odtwarzacze modułów i MP3, gry, `AUDIO:`) wychodzi przez kodek karty, zmiksowany z dźwiękiem Amigi i dekoderem MP3. Cztery tryby *Warp:* w preferencjach AHI, miksowanie 44,1 / 22,05 / 11,025 kHz — niższe częstotliwości odciążają 68k, a karta interpoluje do 44,1 kHz. Na razie tylko odtwarzanie.
- Dźwięk ma **własny kanał w DPRAM**, więc ruch dyskowy i sieciowy go nie przerywa. Tempo miksowania wyznacza karta, dlatego odtwarzanie jest płynne niezależnie od odtwarzacza i ustawienia limitu CPU w AHI.
- **MP3 w każdej częstotliwości:** pliki 48 kHz grały o 8% za wolno i o 1,5 półtonu za nisko; teraz konwersja do 44,1 kHz.
- MHI zgłasza prawdziwy stan dekodera (bitrate, częstotliwość, pozycja) i jest zasilany na żądanie, bez odpytywania.

Grafika

- **Sprzętowy dekoder JPEG** w STM32: `warpjpeg.datatype` dla wszystkich programów używających datatypes oraz loader dla ImageFX. Zdjęcie 12 Mpx dekoduje się w ok. 2 s.

Diagnostyka i instalacja

- Nowe narzędzie **WarpDebug** (polecenia diagnostyczne wydzielone z WarpTool): `qspiStats`, `sysStats` (obciążenie ARM, wątki, pamięć), `asyncDbg`, `netDbg`, test łącza qsBUS.
- Instalator tworzy katalog **WarpApps** na aplikacje Warp.

Aktualizacja: jak przy każdej nowej wersji, pakiet aktualizuje razem obraz FPGA, firmware STM32 i oprogramowanie AmigaOS (w tym `cswarp.library`) — tylko taki komplet jest testowany i wspierany. **Znane ograniczenie:** karta Ethernet USB osiąga pełną prędkość tylko na Warp1260X i Warp7060 (port High-Speed); na pozostałych kartach wysyłanie jest wolne — tam lepszym wyborem jest WiFi.

8. Nowe aplikacje w pakiecie 3.0

WarpDashboard — karta „na żywo”

Podgląd całej karty w czasie rzeczywistym: wskaźniki zegarowe obciążenia procesora ARM, procesora 68k i łącza qsBUS, temperatury oraz przewijane wykresy historii. Pięć stron szczegółów: najbardziej zajęte wątki i pamięć STM32, ruch na łączu z podziałem na rodzaje, odtwarzanie AHI i MP3, ruch WiFi i Ethernetu USB, porty USB i aktywność dysków. Pierwszy raz widać na ekranie Amigi to, co robi „south-bridge” — i dobrze ilustruje to, jak bardzo jest zajęty.

Wymaga ekranu Picasso96 o głębi min. 15 bitów. Tylko czyta liczniki, więc może działać równolegle z WarpDebug. Wskaźnik 68k mierzy jak klasyczne „CPU metery” (zadanie o najniższym priorytecie), więc inne takie mierniki pokażą wtedy 100%.

WarpStressTest — czy ten zegar jest naprawdę stabilny?

Komputer tuż za granicą stabilności często się nie zawiesza — raz na jakiś czas odczyta jeden bit źle albo jednostka FPU zwróci błędny wynik, i pracuje dalej. Dlatego **każdy test sam sprawdza swoje wyniki**, a każdy błąd jest klasyfikowany tak, by raport wskazywał przyczynę:

- **Pamięć:** magistrala danych i adresowa, March C-, testy celowane w L2 (np. seria powtarzanych odczytów trafiających w L2, gdzie działa wczesne potwierdzenie), MOVE16, linie bajtowe. Błąd jest odczytywany ponownie, co odróżnia błąd odczytu (timing) od błędu zapisanych danych, a przekłamane bity są liczone per linia danych D0–D31.
- **CPU i FPU:** CRC, mnożenie/dzielenie, pola bitowe; każda operacja FADD/FSUB/FMUL/FDIV/FSQRT sprawdzana osobno względem emulacji programowej, operacje 80-bitowe bit w bit.
- **Grafika:** blitter FPGA porównywany z modelem każdego piksela, test pamięci VRAM.
- **Łącze qsBUS:** echo ramek, qsDMA do i z DDR3, pełna pętla.

Przykład z testów: Warp7060 podkręcony za mocno najpierw zawodzi w instrukcji FMUL — wszystko inne, łącznie z łączem qsBUS, przechodzi godzinę bez błędu przy 107,5 MHz. Wymaga MUI 3.8; testy grafiki — ekranu Picasso96.

WarpPDFView — zoptymalizowana pod P96 przeglądarka PDF

Przeglądarka dokumentów PDF dla AmigaOS 3.x zbudowana na bibliotece MuPDF, w oknie MUI na ekranie Picasso96. Zakładki (spis treści dokumentu), wyszukiwanie z podświetleniem trafień, tryb pojedynczej strony i tryb ciągły, dopasowanie do szerokości lub strony, zoom 36–200 dpi, wygładzanie krawędzi, kółko myszy. Strony renderuje osobny proces roboczy — okno odpowiada podczas rysowania, a wolny czas idzie na przygotowanie kolejnych stron do pamięci podręcznej.

Wymaga MUI 3.8 i ekranu Picasso96 o głębi min. 15 bitów. Dostępne wersje dla 68060 i 68040.

9. W przygotowaniu: oryginalny obraz Amigi w 50 Hz (genlock)

W jednej z kolejnych wersji 3.x pojawi się aktualizacja scandoublera, czyli układu próbkującego oryginalny obraz z chipsetu Amigi. Pozwoli ona wyświetlić obraz PAL w jego natywnej częstotliwości ok. 50 Hz, a nie przeliczony na 60 Hz. Funkcja jest obecnie w testach wewnętrznych. Jest to jednak problem trudniejszy, niż się wydaje, i warto o nim uczciwie powiedzieć.

Dlaczego nie wystarczy „przełączyć wyjście na 50 Hz”

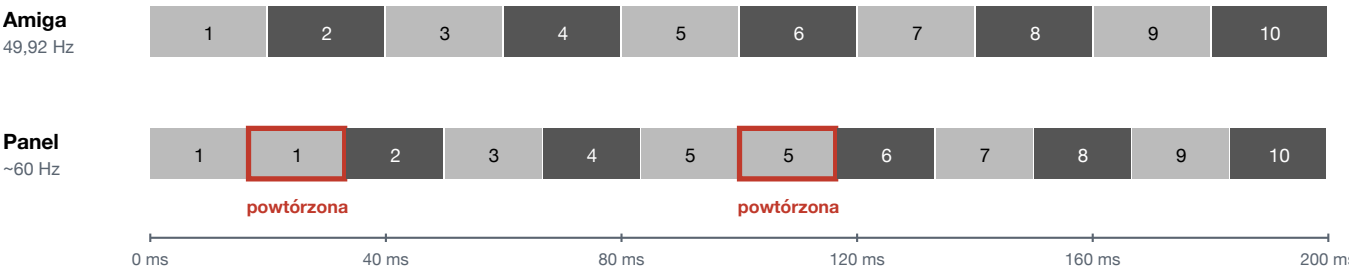
Amiga PAL nie generuje dokładnie 50 Hz, tylko ok. **49,92 Hz**, a dokładna wartość zależy od kwarcu w danym egzemplarzu. Gdyby karta wysyłała obraz z własnego zegara, nawet idealnie równego 50,00 Hz, różnica 0,08 Hz oznaczałaby, że co ok. 12,5 s jedna klatka Amigi musiałaby zostać pominięta albo pokazana dwa razy. Przy płynnym przewijaniu w grze takie szarpnięcie jest dobrze widoczne. Jedynym poprawnym rozwiązaniem jest **pełna synchronizacja (genlock)**: obraz wyjściowy jest taktowany z tego samego zegara co Amiga, a każdej klatce Amigi odpowiada dokładnie jedna klatka na wyjściu HDMI.

Stan prac: synchronizacja działa, monitor i tak przelicza na 60 Hz

Obecny testowy firmware realizuje pełny genlock. Obraz HDMI jest generowany z tego samego zegara co Amiga, w trybie 1280×1024 z niestandardowym timingiem dopasowanym do ramki Amigi. Klatki na wyjściu są w pełni zsynchronizowane, a testowy monitor 27″ 4K poprawnie zgłasza tryb 50 Hz.

Żeby sprawdzić, co naprawdę pokazuje matryca, nagraliśmy ekran w zwolnionym tempie (240 kl./s). Na Amidze działał program testowy, który w każdej klatce zmienia tło na przemian między dwoma odcieniami szarości i przesuwa czerwony pasek o jedną linię. Analiza nagrań (ok. 20 s czasu rzeczywistego, prawie 1000 zmian obrazu) dała jednoznaczny wynik:

- Sygnał z karty jest czysty: klatki przychodzą ściśle naprzemiennie, bez rwania obrazu (tearing).
- **Matryca monitora pracuje jednak z własnym zegarem ok. 60 Hz** (zmierzone 59,975 Hz), niezależnie od trybu 50 Hz na wejściu. Elektronika monitora przelicza klatki: co piąta klatka Amigi jest pokazywana dwukrotnie, czyli ok. 10 razy na sekundę. Pasek, który powinien poruszać się równo, co chwilę zatrzymuje się na jedno odświeżenie i animacja traci płynność.
- Zmiana ustawień monitora nic tu nie zmieniła. Komunikat „50 Hz” w menu monitora mówi tylko o sygnale wejściowym, a nie o tym, z jaką częstotliwością pracuje matryca.



Rys. 4. Co dzieje się w monitorze, który przelicza obraz na 60 Hz: 10 klatek Amigi w ciągu 200 ms trafia na 12 odświeżeń matrycy, więc co piąta klatka jest pokazywana dwa razy (czerwona ramka). Tak właśnie wyglądały nagrania matrycy testowego monitora.

To nie czasy monitorów CRT

Monitor kineskopowy nie miał pamięci klatki ani procesora obrazu. Wiązka elektronów rysowała sygnał dokładnie tak, jak przychodził ze źródła, w jego rytmie. Współczesne monitory i telewizory mają skaler, bufor klatki i rozbudowany procesor obrazu. Wiele matryc pracuje tylko z kilkoma ustalonymi częstotliwościami (najczęściej 60 Hz), a elektronika po swojemu „dopasowuje” każdy inny sygnał. To szerszy problem niż Amiga. Filmy w kinowych 24 kl./s albo w innych niestandardowych częstotliwościach są często przez takie przetwarzanie obrazu wyraźnie zniekształcane: ruch staje się nierówny, a przy niektórych ustawieniach pojawiają się sztuczne klatki pośrednie.

Ten sam sygnał, trzy różne wyniki

Ten sam zsynchronizowany sygnał 1280×1024 / 50 Hz podaliśmy na trzy wyświetlacze:

Wyświetlacz	Wynik
testowy monitor 27″ 4K	Obraz jest, monitor zgłasza 50 Hz, ale matryca pracuje w ok. 60 Hz i przelicza klatki — animacja traci płynność.
nowszy telewizor 65″ 4K	Brak obrazu — telewizor w ogóle nie wyświetlił trybu 1280×1024 / 50 Hz.
starszy telewizor 48″ Full HD	Obraz jest i wyświetlany faktycznie w 50 Hz — płynnie, tak jak powinien.

W skrócie: pracujemy nad tą funkcją i pojawi się ona w jednej z wersji 3.x. Genlock w karcie Warp jest warunkiem koniecznym płynnego obrazu 50 Hz, ale nie wystarczającym — efekt końcowy zależy od konkretnego monitora lub telewizora, i rezultaty mogą być różne. Część współczesnych wyświetlaczy i tak przeliczy obraz na 60 Hz albo nie przyjmie takiego trybu wcale, i karta nie ma na to wpływu. Co ciekawe, nowszy i droższy sprzęt nie oznacza tu lepszego wyniku.

10. Podsumowanie

- Karta Warp ma **jeden procesor aplikacyjny — 68040/68060**. FPGA pełni rolę chipsetu (north-bridge + grafika), a STM32H7 — rolę **south-bridge'a**: koncentratora wejścia/wyjścia, który obsługuje USB, SD, sieć, audio i aktualizacje.
- STM32 jest mikrokontrolerem bez MMU, w całości zajętym obsługą urządzeń w czasie rzeczywistym — dlatego nie przewiduje się uruchamiania na nim programów użytkownika. Jego moc trafia do Amigi w postaci usług: MP3, AHI, JPEG, sieć, dyski.
- Kontroler USB w STM32 nie jest kontrolerem EHCI — procesor sam prowadzi każdą transakcję. Dopracowanie USB wymagało setek poprawek i własnych narzędzi pomiarowych.
- Firmware 3.0 przebudował komunikację Amiga ↔ karta na model asynchroniczny: jedno wolne urządzenie nie blokuje innych, a nowy sterownik AHI ma własny kanał dźwięku. Do tego szybszy Chip RAM, koniec gubionych klawiszy, dekoder JPEG i trzy nowe aplikacje.
- W przygotowaniu jest wyświetlanie oryginalnego obrazu Amigi w natywnych 50 Hz z pełną synchronizacją (genlock). Płynność końcowa zależy jednak także od monitora: wiele współczesnych matryc i tak przelicza obraz na 60 Hz.

Projekt jest rozwijany od 2019 roku: ok. 800 commitów w repozytorium firmware/FPGA i ponad 300 w repozytorium oprogramowania AmigaOS. Sam cykl 2.027 → 2.282 to ok. 145 godzin pracy, ok. 41 tys. linii nowego i zmienionego kodu i ok. 960 cykli kompilacji i wgrania firmware.